Shell Scripting Guidelines

Change History

Purpose

This document provides guidance for developing maintainable, secure, and reliable shell scripts used in operational, automation, and administrative workflows. These guidelines are intended to improve readability, consistency, troubleshooting, and long-term supportability.

Audience

While the content is technical (for programmers, developers, IT architects, etc.), it should be available to the entire project team. Contact the admin if you have questions or recommend additional information.

When to Use Shell Scripts

Good candidates for shell scripts:

- System administration
- File manipulation
- Automation
- Deployment orchestration
- Monitoring tasks
- Scheduled maintenance tasks
- Batch processing

Consider another language when:

- Application logic becomes complex
- Data structures become sophisticated
- Extensive error handling is required
- The script exceeds several hundred lines
- Long-term maintainability becomes difficult

Core Principles

1. Keep scripts simple.
2. Fail early and clearly.
3. Validate all inputs.
4. Make scripts self-documenting.

5. Prefer readability over brevity.
6. Design for maintainability.

Script Design & Development

Script Structure

Typical layout:

```
#!/usr/bin/env bash

set -euo pipefail

# Constants
# Variables
# Functions
# Main Logic
```

Script Header

Include basic information at the beginning of the script.

Suggested information:

- Script name
- Purpose
- Author
- Version
- Usage example
- Dependencies

Example:

```
#!/usr/bin/env bash

# Name: backup.sh
# Purpose: Perform nightly backup
# Version: 1.0
# Usage: backup.sh <source> <destination>
```

Functions

Use functions to organize reusable logic and improve readability.

Guidelines:

- Keep functions focused on a single responsibility.
- Use descriptive function names.
- Avoid excessively large functions.

- Document complex functions when necessary.
- Return meaningful exit codes.

Example uses:

- File validation
- Logging
- Error handling
- Data processing

Variables & Naming

Use consistent naming conventions to improve readability and maintainability.

Guidelines:

- Use descriptive variable names.
- Avoid single-character variable names except in short loops.
- Use uppercase for constants.
- Use lowercase for local variables and functions.
- Quote variable expansions unless there is a specific reason not to.

Examples:

```
BACKUP_DIR="/data/backups"
server_name="web01"
```

Avoid:

```
d="/data/backups"
x="web01"
```

Configuration Management

Prefer configuration files, environment variables, or script parameters over hard-coded values.

Avoid embedding environment-specific settings directly in scripts whenever possible.

Examples of values that should ideally be configurable rather than hard-coded:

- Server names
- Database names
- Directory paths
- Retention periods
- Environment names (DEV, TEST, PROD)

- Network endpoints

For example, instead of writing:

```
BACKUP_DIR="/data/backups"
RETENTION_DAYS=30
SERVER="prod-server-01"
```

... a script might accept those values as parameters or read them from a configuration file.

Comments & Documentation

Shell scripts should be easy to understand, maintain, and troubleshoot by someone other than the original author.

Document Script Purpose

Every script should clearly communicate:

- What the script does
- When it should be used
- Required inputs and parameters
- Expected outputs
- External dependencies
- Any prerequisites or assumptions

This information may be included in the script header, associated documentation, or team wiki.

Comment Why, Not What

Comments should explain the reason for complex logic or unusual behavior rather than restating what the code already does.

Prefer:

```
# Retry because the remote service occasionally returns transient
errors
```

Instead of:

```
# Increment counter
counter=$((counter + 1))
```

The code already shows that the counter is being incremented.

Keep Documentation Current

Documentation and comments should be updated whenever script behavior changes.

Outdated documentation can be more harmful than having no documentation at all.

Document Non-Obvious Logic

Provide additional explanation when:

- Business rules affect processing
- Workarounds exist for system limitations
- Special handling is required
- External system dependencies influence behavior

Provide Usage Information

Scripts intended for operational use should provide clear usage instructions when incorrect parameters are supplied.

Example:

```
Usage: backup.sh <source_directory> <destination_directory>
```

Avoid Excessive Comments

Well-structured scripts with meaningful variable names and functions generally require fewer comments.

Use comments to clarify intent, assumptions, and design decisions rather than describing every line of code.

Document Operational Impact

Where applicable, document:

- Expected runtime
- Resources affected
- Files created or modified
- Services restarted
- Potential risks of execution

Example:

```
Operational Notes:
- Expected runtime: 10-15 minutes
- Creates backup files in /data/backups
- Requires network access to backup server
- Does not restart services
```

Runtime Reliability

Input Validation

Input validation should be performed as early as possible in script execution. Scripts should verify that all required inputs, parameters, files, directories, and dependencies are valid before processing begins.

Validating input helps prevent unexpected failures, improves troubleshooting, and provides users with clear feedback when incorrect information is supplied.

Validate Required Arguments

Ensure that required command-line arguments have been provided.

Examples:

- A **filename** was supplied when a **file** is required.
- A **server name** was provided.
- A **user account** name was specified.

If required information is missing, display a usage message and exit gracefully.

Validate Argument Format

Verify that input values match the expected format.

Examples:

- **Numeric** values contain only **numbers**.
- **Email addresses** follow an acceptable **format**.
- **IP** addresses are **valid**.
- **Dates** conform to the required **format**.

Do not assume that user input is correct.

Validate File & Directory Existence

Before reading from or writing to a file, verify that the file or directory exists.

Examples:

- Confirm that **input files** are present.
- Confirm that **target directories** exist.
- Verify that **configuration files** can be located.

🟦 Provide a clear error message if a required resource cannot be found.

Validate Permissions

Verify that the script has the required **permissions** before performing operations.

Examples:

- **Read access** to input files.
- **Write access** to output directories.
- **Administrative privileges** when required.

🟦 Fail early if the required permissions are not available.

Validate External Dependencies

Check that required commands, tools, and applications are installed and available.

Examples:

- awk
- sed
- grep
- curl
- ssh

🟦 Notify the user if a required dependency is missing.

Validate Environment Assumptions

Confirm that expected environmental conditions exist before proceeding.

Examples:

- Required **environment variables** are defined.
- **Network resources** are reachable.
- **Required services** are running.
- Sufficient **disk space** is available.

Handle Invalid Input Gracefully

When validation fails:

- Display a clear and actionable error message.
- Explain what was expected.

- Exit with an appropriate return code.
- Avoid continuing execution with invalid data.

Example:

Instead of:	Provide
Error	Input file '/data/input.csv' was not found. Verify the file path and try again.

Common Validation Checks

Validation Type	Example
Required argument	Did the user supply a filename?
File exists	Does /tmp/input.txt exist?
Directory exists	Is /var/log/app present?
Numeric value	Is retention period a number?
IP address	Is the server address valid?
Host reachable	Can the server be pinged?
Command available	Is curl installed?
Permission check	Can the script write to the destination?
Disk space	Is there enough space for backup files?
Environment variable	Is $JAVA_HOME defined?

Error Handling

```bash
</> Bash
set -euo pipefail
```

and

```
</> Bash
trap cleanup EXIT
```

The **trap** command is useful because it allows a script to catch asynchronous operating system signals or internal shell events and execute cleanup code before exiting.

Exit Codes

Scripts should return meaningful exit codes to indicate success or failure.

Common conventions:

Exit Code	Meaning
0	Success
1	General error
2	Invalid arguments
126	Command cannot execute
127	Command not found

Benefits:

- Supports automation.
- Enables monitoring tools.
- Simplifies troubleshooting.

Temporary Files

When creating temporary files:

- Use secure temporary file creation methods.
- Remove temporary files when processing completes.
- Use cleanup handlers where appropriate.

Example:

```
trap cleanup EXIT
```

Logging

Scripts should generate meaningful log messages that help operators understand what the script is doing and assist with troubleshooting.

Recommended practices:

- Log major processing steps.
- Log errors with sufficient context.
- Include timestamps when appropriate.
- Differentiate informational, warning, and error messages.
- Avoid logging sensitive information such as passwords, tokens, or private keys.
- Ensure log messages are clear and actionable.

Examples:

```
INFO: Starting backup process
INFO: Processing 25 files
WARNING: Configuration file not found. Using defaults.
ERROR: Unable to connect to database server.
```

Quality & Operations

Security Considerations

- Never store passwords in scripts.
- Validate user input.
- Quote variable expansions.
- Avoid eval.
- Use least privilege.

Performance Considerations

Examples:

Avoid	Prefer
</> Bash for file in $(ls)	</> Bash for file in *

Avoid repeatedly calling external commands inside loops when possible.

Testing & Validation

- Test success paths.
- Test failure paths.
- Test boundary conditions.
- Test with unexpected input.

Tooling

Recommended tools:

- ShellCheck
- Git
- CI/CD pipeline validation

Change Management / Source Control

All production scripts should:

- Be stored in version control.
- Be reviewed before deployment.
- Include change history through commit messages.
- Follow team branching and release practices.

Portability

If scripts may run across different Linux distributions or UNIX environments:

- Avoid distribution-specific commands when possible.
- Verify shell compatibility requirements.
- Document required operating system versions.
- Document external dependencies.

Common Pitfalls & Examples

Common Anti-Patterns

Unquoted Variables

Bad	Good
`</> Bash` `rm $file`	`</> Bash` `rm "$file"`

Parsing Is Output

Bad	Good
`</> Bash` `for f in $(ls)`	`</> Bash` `for f in *`

Ignoring Exit Codes

Bad	Good

`</> Bash`
`cp file1 file2`
	`</> Bash`
	`if ! cp file1 file2; then`
	`    echo "Copy failed"`
	`fi`

Example Script Template

> **Note**: This template is intended to illustrate recommended scripting practices. Actual implementations may require additional logging, validation, error handling, or functionality depending on the use case.

See Checklist below.

```bash
#!/usr/bin/env bash

set -euo pipefail

################################################################################
# Name: example-script.sh
# Purpose: Demonstrate recommended shell scripting practices
# Usage: example-script.sh <input_file>
################################################################################

readonly SCRIPT_NAME="$(basename "$0")"

log_info() {
    echo "[INFO] $*"
}

log_error() {
    echo "[ERROR] $*" >&2
}

usage() {
    echo "Usage: $SCRIPT_NAME <input_file>"
}

validate_input() {
    if [[ $# -ne 1 ]]; then
        usage
        exit 2
    fi

    local input_file="$1"

    if [[ ! -f "$input_file" ]]; then
        log_error "Input file not found: $input_file"
        exit 1
    fi

    if [[ ! -r "$input_file" ]]; then
        log_error "Input file is not readable: $input_file"
        exit 1
```

```
    fi
}

cleanup() {
    log_info "Performing cleanup tasks"
}

main() {
    local input_file="$1"

    log_info "Starting script"

    validate_input "$input_file"

    log_info "Processing file: $input_file"

    # Main processing logic goes here

    log_info "Script completed successfully"
}

trap cleanup EXIT

main "$@"
```

Checklist

Before Production Use	Verify
Script header included	✓
Input validation implemented	✓
Error handling implemented	✓
Logging implemented	✓
Variables quoted appropriately	✓
ShellCheck warnings resolved	✓
Exit codes documented	✓
Tested with invalid input	✓
Stored in source control	✓

References

- Internal Standards
- Team Conventions
- ShellCheck
- Bash Manual

- Content History

Content Control

Status: PUBLISHED

Role	Name	Date
Author	Polly Pocket	02-Feb-2025
Editor	Sharon Hawkins	04-Feb-2025
Reviewer	Dexter Thecat	10-Feb-2025

Change History

Name	Date	Description
Polly Pocket	03-Feb-2025	Added script sample template.
Suzy Queue	04-Feb-2025	Added sample scripts as code snippets throughout.